



© *Philippe Roose – 2000, 2003*
IUT Informatique de Bayonne

Table des matières

1	<i>Qu'est-ce donc ?</i>	3
1.1	Le format ShockWave Flash (SWF)	3
1.2	Le format Scalable Vector Graphics (SVG)	3
2	<i>Le vocabulaire</i>	4
3	<i>Interface de travail</i>	5
3.1	Détail des principaux éléments	5
4	<i>Les palettes</i>	5
4.1	Palette Info	6
4.2	Autres palettes	6
5	<i>Exemples simples</i>	7
5.1	Interpolation de forme (exo1 fla)	8
5.2	Interpolation de mouvement (exo2 fla)	9
5.3	Guide de mouvement (exo3 fla)	9
5.4	Création de masques simples (exo4 fla)	10
5.5	Création de masques animés (exo5 fla)	10
5.6	Boutons fixes (exo6 fla)	11
5.7	Boutons animés (exo7 fla)	12
5.8	Boutons à bascule (exo8 fla)	12
6	<i>Les actions scripts</i>	13
7	<i>Propriétés des clips</i>	13
8	<i>Détection des collisions (exo9 fla)</i>	14
9	<i>Détection de « clics souris » et duplication de dessins (exo10 fla)</i>	15
10	<i>Déplacement interactifs de clips (exo11 fla)</i>	16
11	<i>Déplacement dans un clip (exo12 fla)</i>	17
12	<i>Gestion de l'interactivité</i>	18
12.1	Clavier (exo13 fla)	18
12.2	Souris – Coordonnées (exo14 fla)	19
12.3	Souris – Drag and Drop (exo15 fla)	19
13	<i>Composants d'interface (exo16 fla)</i>	19
14	<i>Gestion des variables par fichiers</i>	20
15	<i>Conclusion</i>	21

1 Qu'est-ce donc ?

Flash est un logiciel de la société Macromédia qui permet de rendre vos pages web à la fois plus interactives mais également plus vivantes.

En effet, Flash permet d'ajouter des animations interactives, des plus simples aux (presque) plus compliquées sans être un as de la programmation. De plus, au fil des versions, Flash s'est enrichi d'un langage qui permet maintenant de réaliser des formulaires de saisie, d'interroger des bases de données et de réagir en conséquence, ...

Au fil des années, Flash est devenu plus qu'un simple logiciel de création d'animations, et près de ½ million de développeurs à travers le monde l'utilisent.

Des outils comme Flash, il y en a eu plusieurs. Ce qui a fait son succès, c'est qu'au delà des caractéristiques de l'outil lui-même, les concepteurs ont produit des players gratuits, légers qui ont fait qu'ils ont été intégrés d'office à la plupart des navigateurs du marché. De plus, et c'est peut-être son principal avantage, à l'époque où on ne parlait pas encore d'ADSL, Flash produisait des fichiers particulièrement compacts (quelques octets ou Ko), avec lesquels il était possible de réaliser énormément de choses (avantages du vectoriel).

Le plug-in Flash est présent sur 98% des navigateurs actuels (installé en standard avec le navigateur).

Le format de fichier *swf* est peu à peu devenu un standard concernant les fichiers d'animations vectoriels. En fait, on peut le considérer depuis 2000, c'est à dire depuis que même ses concurrents directs (Live Motion d'Adobe) utilisent le même format.

1.1 Le format ShockWave Flash (SWF)

Le format *swf* et flash sont 2 choses différentes. En effet, Flash combine d'autres choses comme le langage *actionScript* basé sur la norme ECMAScript (dont Javascript de Netscape & JScript de Microsoft dérivent également).

Plusieurs formats sont disponibles :

- **Formats d'édition** : *.fla* (-> version 5) et *.flv (MX)*.

Le *.fla* permet de traiter et de monter les fichiers importés (*images, sons, textes, dessins vectoriels, ...*), il permet de les animer et d'éventuellement les programmer. Le *.flv* permet en outre de traiter la vidéo (*montage, habillage, compression*).

- **Formats de publication** : *.swf* et *.exe*

Le *.swf* est un format ouvert (public) de publication sur le web. Il nécessite d'être « encapsulé » dans du code HTML pour être utilisé par un plug-in intégré au navigateur.

Enfin il est également possible de générer du code autonome pour PC dans la version « off-line » du plug-in.

1.2 Le format Scalable Vector Graphics (SVG)

Néanmoins, depuis le 5/9/2001, le consortium W3C qui s'occupe des normes liées à internet a défini et publié une recommandation concernant le format SVG 1.0 (*Scalable Vector Graphics*), publication concernant les animations et de dessin vectoriels et fondée sur XML.

La spécification publiée par le W3C signifie son adoption, ce qui veut dire que les publications de ce format sont dorénavant stables. Au delà du W3C, ce format possède le support de nombreux industriels.

SVG n'est pas une simple alternative à SWF, il permet en outre la réutilisation de documents aux formats CSS, XSL, les liens XML-links...

Étant issu du XML, le format SVG est textuel et donc facilement éditable. Du fait de son origine XML^{ienne} (!) même, il propose une grande facilité d'interopérabilité, de recherche, d'indexation, et permet de manipuler des objets d'autres langages. Il peut également être généré à l'aide de Java.

Enfin, ce format étant ouvert, ceci lui procure une certaine pérennité et ouverture.

Attention, la naissance de ce format signifie par la mort de Flash, loin de là. Premièrement, il faudra qu'il soit adopté par la communauté et que son plug-in soit intégré dans les navigateurs. Ensuite, il est tout à fait envisageable que Macromédia Flash propose d'enregistrer ses animations dans ce format..

Ce faisant, Flash est devenu le standard des animations vectorielle pour le Web comme le sont devenus Director ou ToolBook pour les CD-Roms. Les concepteurs de sites web utilisent Flash pour créer des interfaces de navigation, re-dimensionnables et extrêmement compactes, des illustrations techniques, de longues animations et d'autres effets pour leurs sites.

À outil spécifique, vocabulaire spécifique !

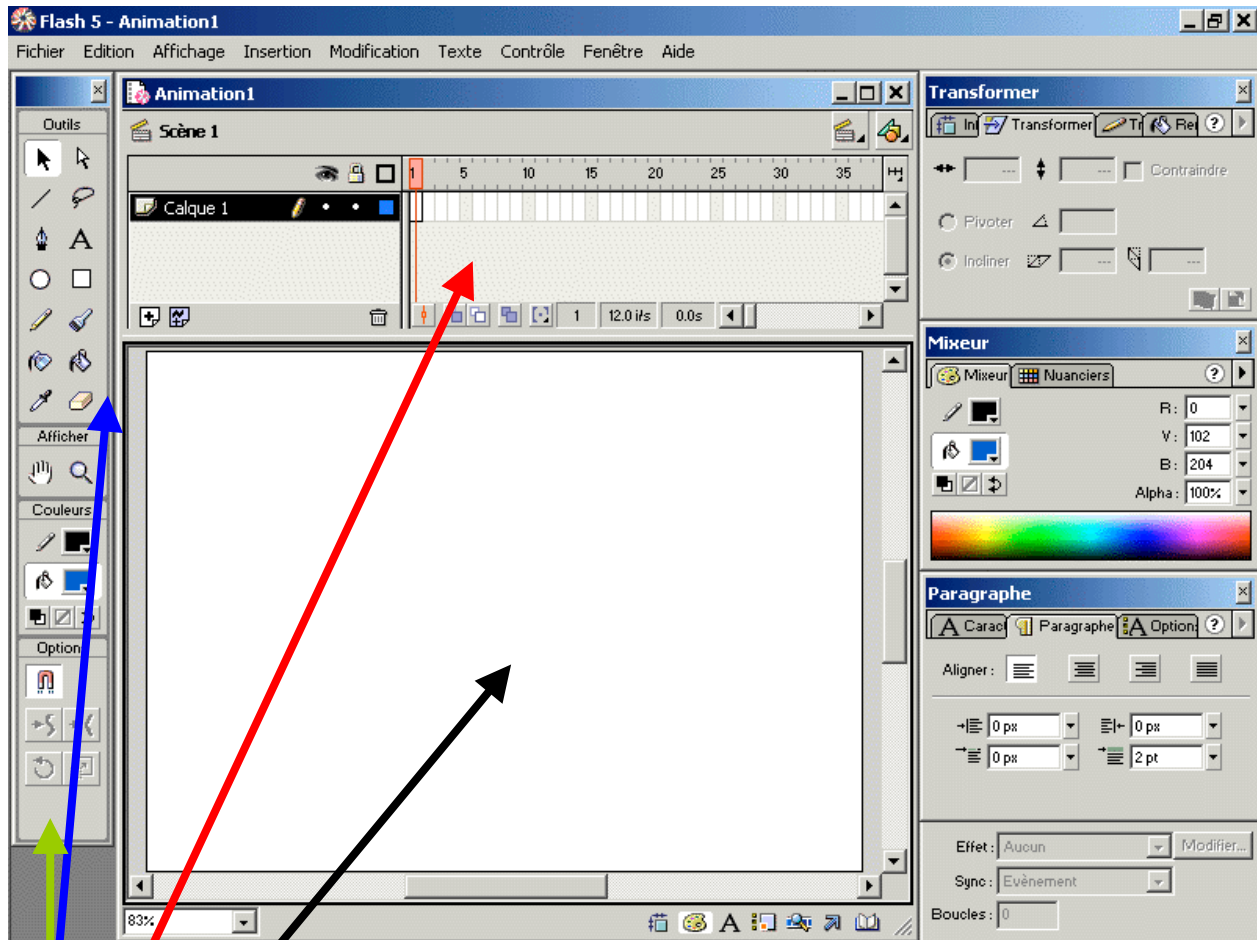
2 Le vocabulaire

- **Scène** : la scène représente ce que verrait un réalisateur au travers de sa caméra. Une animation Flash peut avoir plusieurs scènes, cela permet de pré-charger une scène pendant que l'internaute en voit une autre, ou de structurer son animation de manière organisée.
- **Scénario** : le scénario avec la **grille** ainsi que les **calques** vont permettre le positionnement des composants. Le scénario contient la totalité des informations du réalisateur, la manière d'organiser les acteurs, les fonds, qui fait quoi, qui bouge, où, dans quel ordre, à quelle vitesse, etc. Il indique également les interactions possibles et définit les étiquettes de navigation, de contrôle, etc. C'est la fenêtre qui contrôle tout. En fait, le scénario représente la pellicule avec l'ensemble des images à afficher.
- **La bibliothèque** : elle contient la liste des éléments constituant une animation (Ctrl. + L).
- **Frame** : En Flash, une image s'appelle une **frame**.
- Une **image clé** (rond noir) est une frame dans laquelle se situent un ou plusieurs dessins. Quand une image clé est vide (pas de dessin), elle est représentée par un rond vide. Une image clé servira à créer une interpolation (entre 2 images clés).
- **Les calques** : L'**œil** permet de cacher un calque lorsqu'il devient parfois difficile de le sélectionner. **Calque** : C'est une couche transparente sur laquelle il est possible de dessiner ou d'inclure des symboles. Le **cadenas** verrouille le calque, il reste visible, mais n'est pas modifiable. Le **contour** permet d'afficher le contour des dessins du calque. Cette option est intéressante lorsque l'on place des composants invisibles.
- Au départ, on place l'ensemble des éléments constituant la **scène**. L'étape suivante va consister à les transformer en composants/symboles réutilisables.
- **Symbole** : C'est un objet de Flash. Il existe 3 types d'objet : graphique, bouton et clip.
- **Occurrence** : C'est une image d'un symbole sur une scène.

Attention, il ne faut pas confondre symbole et occurrence. Un symbole est stocké dans la bibliothèque (c'est en gros, le dessin que l'on transforme en ... symbole ! A chaque fois que nous puisons ce symbole pour le mettre sur la scène, nous créons une occurrence. Chaque occurrence (distincte d'une autre) d'un même symbole peut être modifiée par les outils de rotations, colorisation, effet de palettes, ... Attention, la modification d'un symbole modifiera toutes ses occurrences, pas l'inverse.

- **Interpolation** : C'est la création d'une image à partir de deux images clés. Il existe deux types d'interpolations : les interpolations d'animation et les interpolations de forme (qui ne doivent pas contenir de symboles).

3 Interface de travail



3.1 Détail des principaux éléments

- La **boîte à outils** contient tous les outils qui vous permettent de dessiner, sélectionner et modifier vos animations.
- Dans la **boîte option** s'affichent les options liées à l'outil sélectionné.
- La **fenêtre scénario** affiche tous les calques de vos animations. Elle sert à mettre en scène les éléments de vos créations.
- La **surface de travail** permet de dessiner et de voir les animations.

4 Les palettes

L'interface de Flash compte aussi plusieurs palettes permettant de produire des effets spéciaux dans vos animations.

En voici quelques-unes (*les deux suivantes sont les plus importantes*) :

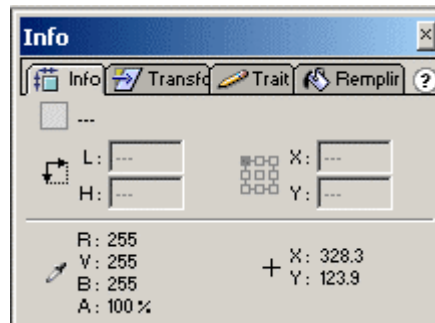
- **Bibliothèque** : Affiche tous les éléments de l'animation.

Pour l'ouvrir : Cliquez sur Fenêtre > Bibliothèque (F11).

- **Info** : Affiche et permet de modifier les informations, comme la hauteur, la longueur et la position des formes géométriques.

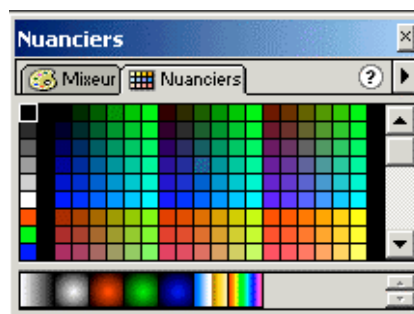
Pour l'ouvrir : Cliquez sur Fenêtre > Panneaux > Info, (Ctrl+Alt+I).

4.1 Palette Info

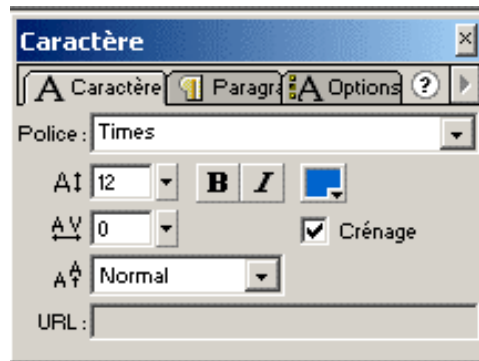


- **Transformer** : Permet de re-dimensionner, pivoter ou incliner les formes géométriques.
Pour l'ouvrir : Cliquez sur Fenêtre > Panneaux > Transformer.
- **Trait** : Permet de modifier l'apparence des traits.
Pour l'ouvrir : Cliquez sur Fenêtre > Panneaux > Trait.
- **Remplir** : Permet d'ajouter un dégradé au remplissage.
Pour l'ouvrir : Cliquez sur Fenêtre > Panneaux > Remplir.

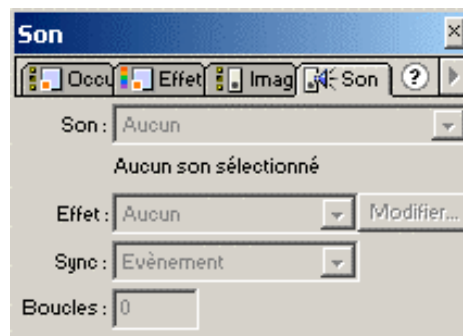
4.2 Autres palettes



- **Aligner** : Permet d'aligner les objets sélectionnés dans la surface de travail.
Pour l'ouvrir : Cliquez sur Fenêtre > Panneaux > Aligner (Ctrl+K).
- **Mixeur** : Permet de composer la couleur de tous les éléments d'une animation.
Pour l'ouvrir : Cliquez sur Fenêtre > Panneaux > Mixeur.
- **Nuanciers** : Permet de modifier la couleur de tous les éléments d'une animation. Permet aussi d'enregistrer les couleurs et dégradés que vous avez créé.
Pour l'ouvrir : Cliquez sur Fenêtre > Panneaux > Nuanciers.



- **Caractère** : Permet de modifier le format de la police.
Pour l'ouvrir : Cliquez sur Fenêtre > Panneaux > Caractère.
- **Paragraphe** : Permet de mettre le texte en page
Pour l'ouvrir : Cliquez sur Fenêtre > Panneaux > Paragraphe.
- **Option de texte** : Permet le texte dynamique ou la saisie de texte.
Pour l'ouvrir : Cliquez sur Fenêtre > Panneaux > Option de texte.



- **Occurrence** : Permet de modifier les occurrences d'une animation.
Pour l'ouvrir : Cliquez sur Fenêtre > Panneaux > Occurrence (Ctrl+I).
- **Effet** : Permet de modifier la luminosité, la teinte et la transparence d'une image.
Pour l'ouvrir : Cliquez sur Fenêtre > Panneaux > Effet.
- **Image** : Permet de nommer une image et de lui donner une interpolation.
Pour l'ouvrir : Cliquez sur Fenêtre > Panneaux > Image.
- **Son** : Permet de modifier le son.
Pour l'ouvrir : Cliquez sur Fenêtre > Panneaux > Son.
- **Scène** : Permet de sélectionner les scènes d'une animation.
Pour l'ouvrir : Cliquez sur Fenêtre > Panneaux > Scène.

5 Exemples simples

Lorsque l'on désire réaliser des mouvements dans une scène (déplacement, agrandissements, rotations, transparences, ...), il est nécessaire de passer par ce que l'on appelle des interpolations. Il en existe de 2 sortes :

- Les interpolations de mouvement (fonctionnent avec des occurrences).
- Les interpolations de forme (fonctionnent uniquement avec des formes vectorielles).

Une interpolation se réalise entre 2 images clés, mais ce qui n'empêche pas de réaliser des interpolations plus complexes en utilisant plusieurs images clés successives.

5.1 Interpolation de forme (exo1 fla)

Notre premier exemple d'application (ex1 fla) va consister à transformer un rond rouge en carré de même couleur.



Dans la première case (frame 1) du scénario, on va dessiner un rond rouge en utilisant la boîte à outils. A la frame 20, nous allons insérer une image clé (F6 ou Insertion/image clé), supprimer le rond rouge et dessiner un carré à la place.

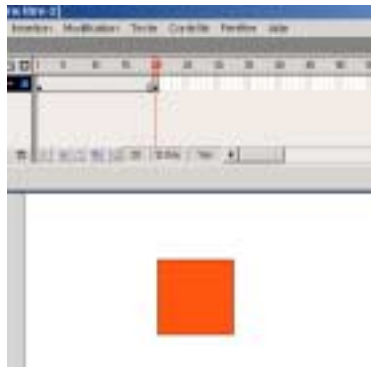


Figure 1 : Interpolation de forme

En cliquant ensuite de nouveau sur la première image, nous allons voir dans les propriétés la rubrique « interpolation » et sélectionner « Forme ».

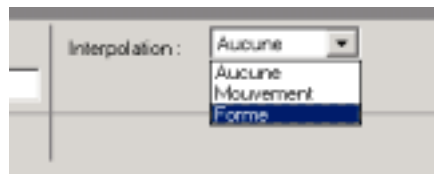


Figure 2 : Interpolation de forme

Les images intermédiaires se colorisent en vert pale. Et hop, le tour est joué, il ne reste plus qu'à lire l'animation (Appuyez sur Entrée ou Contrôle/Lire).

Attention : contrairement aux interpolation de mouvement que nous allons voir maintenant, il ne fait pas créer de symbole pour réaliser une interpolation de forme.

5.2 Interpolation de mouvement (exo2 fla)

On crée à la première frame un rond rouge que l'on va transformer en symbole graphique (Ctrl+F8 après l'avoir sélectionné ou Insertion/Convertir en symbole). Il va se trouver encadré de bleu.

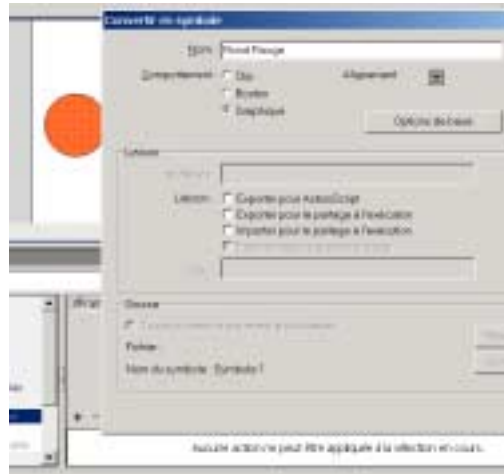


Figure 3 : Interpolation de mouvement

On va ensuite à l'image 15, on insère une image, notre symbole est recopié, on va le déplacer à droite. Idem à l'image 30 mais à droite et en bas. On va ensuite créer une interpolation de mouvement en se plaçant sur l'image clé de départ. L'interpolation ira jusqu'à l'image clé suivante. On reproduira la même chose pour le second segment. On doit voir maintenant que les images reliées par un fond bleu et une flèche en trait plein (si elle est en pointillé, y'a un problème ! !). Il ne reste plus qu'à tester (Ctrl + Entrée).

Il est possible de simuler des accélérations en utilisant la rubrique « accélération » dans les propriétés de l'animation. Il est également possible de sélectionner des rotations automatique (gauche ou droite) entre la position initiale et celle d'arrivée.

5.3 Guide de mouvement (exo3 fla)

Il est parfois nécessaire de réaliser une animation non pas rectiligne mais selon un tracé un peu complexe (une piste de circuit, ...). On va pour cela utiliser les guides de mouvements.

On va recréer notre rond rouge et le retransformer en symbole. On va ensuite créer une image clé à la destination de notre mouvement (frame 30) puis insérer un guide (Insertion/Guide de mouvement). A la frame 1, on dessine (outil dessin à main de la boîte à outils par exemple) la trajectoire en partant du centre du cercle, puis à la frame 30, on insère une image clé ce qui aura pour effet de recopier le chemin tracé. On va ajuster le centre du rond rouge sur la fin de la trajectoire dessinée. Il ne reste plus qu'à générer l'interpolation de mouvements du cercle.

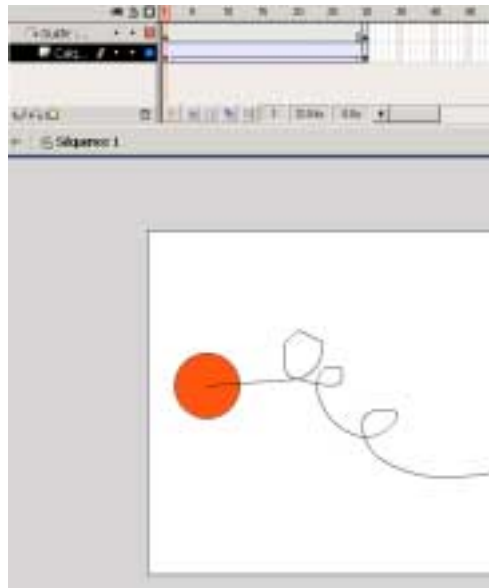


Figure 4 : Guide de mouvement

5.4 Création de masques simples (exo4 fla)

On va importer une image (Fichier/Importer) et afin de s'y retrouver, renommer la calque correspondant en « image ». On va ensuite créer un second calque appelé « Rond » et y insérer un rond. On va dire que ce calque est particulier en lui donnant un comportement de masque (bouton droit sur le calque/propriétés/masque). Attention de bien respecter l'ordre de création du masque/spécification du calque.



Figure 5 : Création du masque



Figure 6 : Masque résultat

5.5 Création de masques animés (exo5 fla)

Nous allons dessiner un rectangle noirs et y insérer un texte quelconque (couleur du texte différente du rectangle ! !). Nous allons ensuite recopier cette image (F5 ou Insertion/image) sur 40 frames. Ajoutons un calque sur lequel nous dessinons un cercle blanc que nous transformons en symbole et que nous placerons au début du rectangle. En frame 40, nous ajoutons ce symbole mais à l'autre bout du rectangle. Nous créons l'interpolation de mouvement (ce qui explique nous nous ayons crée un symbole). Puis, nous signalons que ce calque est un masque.

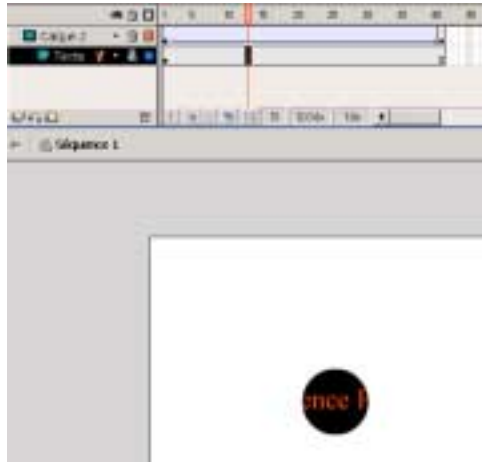


Figure 7 : Masque animé

5.6 Boutons fixes (exo6 fla)

Un bouton fixe ne change d'aspect lorsque l'on clique dessus. On va dessiner un rond et y insérer un texte. On va ensuite transformer ce dessin en symbole de type bouton.

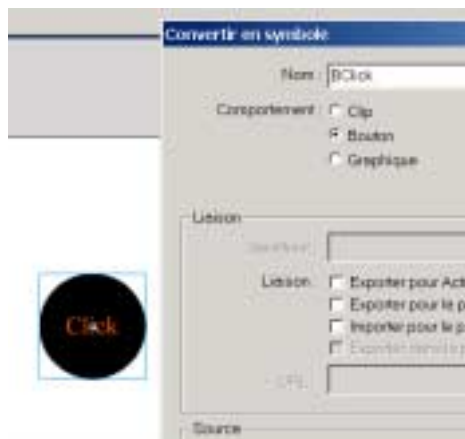


Figure 8 : Création d'un bouton

On va entrer en mode d'édition. Pour cela, on va afficher la bibliothèque des symboles (Ctrl+L/F11) et double cliquer sur le symbole nouvellement créé. On peut maintenant travailler sur les différents aspects qu'il doit prendre lors de son utilisation.

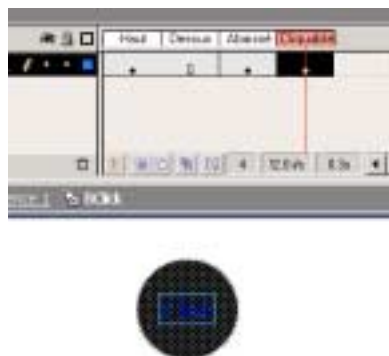


Figure 9 : États d'un bouton

5.7 Boutons animés (exo7 fla)

Il est également possible d'avoir des boutons qui s'animent en permanence, lors d'une prise de focus, ou lors d'un clic. Pour cela, ils doivent contenir une animation.

Nous allons créer une animation : Insertion/Nouveau Symbole/Clip que nous nommerons *animB*. Nous sommes maintenant dans sa fenêtre d'édition et allons créer une animation par interpolation (attention si vous faites une interpolation de mouvement)...il faut un symbole. L'animation devra être un rond qui grossit et qui revient à sa taille initiale (il faudra donc 3 images clés).

Créons un nouveau symbole de type bouton cette fois. Dans sa fenêtre d'édition, nous allons mettre un dessin dans la rubrique haut, et insérer une image clé (F6) dans la rubrique dessus pour y insérer notre animation (par *Drag and Drop*).

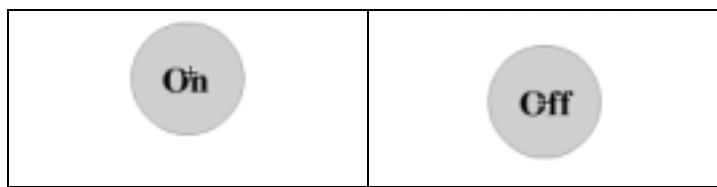
Haut	Dessus	Abaissé	Cliquable
Vue « hors focus »	Vue avec focus	Vue lors d'un clic	Zone cliquable

Dans le cas où la rubrique « abaissée » est utilisée (dessin du bouton « appuyé »), on parlera de boutons réactifs. Il est également possible de faire des boutons transparents en y insérant des dessins/animations aux formes non géométriques, mais en remplissant la rubrique cliquable, il sera possible de leur définir un rectangle d'action.

5.8 Boutons à bascule (exo8 fla)

Les boutons à bascule ont la particularité de rester dans l'état « cliqué » et de ne revenir à l'état initial que lors d'un second clic dessus.

Nous allons en écrire un ce qui nous permettra également d'écrire nos premières lignes de code. Dans un premier temps, nous allons créer 2 boutons On et Off.



Nous créons ensuite un clip avec le bouton « On » en frame 1 et « Off » en 10. En frame 1 nous allons également associer une action « stop() » qui s'obtient avec le menu contextuel sur la première frame. On verra un petit « a » d'écrire dans la timeline.

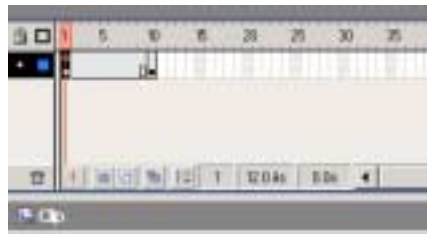


Figure 10 : Création d'une action

Nous allons ensuite sélectionner le bouton « On » et lui associer le code suivant :

```
on (press) {  
    gotoAndStop(10);  
}
```

De manière symétrique, au bouton « Off » :

```
on (press) {  
    gotoAndStop(1);  
}
```

Il ne reste plus qu'à placer notre bouton sur la scène et à tester l'animation (Ctrl+Entrée), qui ne fera pas grand chose puisque nous n'avons pas associé d'action au « clic ».

6 Les actions scripts

Depuis la version 5 de Flash, les actions scripts permettent d'intégrer des aspects de programmation à Flash lui permettant une meilleure gestion de l'interactivité au sein de sites Web. Il est par exemple possible de l'interfacer avec du PHP.

D'un point de vue philosophie et syntaxe, les actions scripts se programment un peu à la manière de Javascript, donc avec une forte coloration d'orienté-objet.

Les méthodes se retrouvent dans le panneau Action (*Fenêtre/action ou Ctrl + Alt + A*). Deux modes sont possibles : le mode normal et le mode avancé.

Afin de faciliter la navigation dans le code, l'éditeur proposé permet de facilement identifier :

- Les mots clés et identifiants prédéfinis seront en bleu,
- Les propriétés des objets en vert,
- Les commentaires en magenta,
- Les chaînes de caractères en gris.

Dans le mode normal, la programmation se fait en sélection la méthode et/ou les attributs désirés dans la fenêtre action. Une fois la sélection réalisée, vous serez placé dans la fenêtre de code en attendant de rentrer les valeurs nécessaires.

Dans le mode expert, vous êtes entièrement libre d'écrire ce que vous voulez (à vos risques et périls !). Il est également possible de saisir le code dans un éditeur de texte quelconque et ensuite de l'intégrer en utilisant la méthode **include**.

Exemple :

```
#include "monfichier.as" (as pour Action Script).
```

Comme tout langage de programmation qui se respecte, Action Script possède un débogueur (Contrôle/Déboguer l'animation) qui permettra de visualiser les états de vos variables à la fin, mais qui permettra surtout de visualiser leurs valeurs au cours de l'exécution de votre animation Flash.

7 Propriétés des clips

Un certain nombre de propriétés des clips permettent d'obtenir des rendus très intéressants qui vous faciliteront la vie. Ces propriétés s'utilisent dans les Actions-Scripts (AS) de la manière suivante :

NomDuClip.propriété = valeur

Les propriétés sont les suivantes :

- **Alpha** : la valeur est comprise entre 0 et 100. 50 permettra par exemple de réduire l'opacité de 50%.
- **Rotation** : s'exprime en degrés et permet d'appliquer une rotation à votre clip.
- **Visible/Invisible** : c'est un booléen, 1 ou 0, true ou false.
- **Width/Height** : hauteur et largeur du clip en pixels.
- **Xscale/Yscale** : étire en hauteur/largeur le clip. Pour l'agrandir de 1,5 fois, on mettra la valeur 150.
- **X/Y** : Position du clip en pixels dans la fenêtre.

8 Détection des collisions (exo9 fla)

Très utile lorsque l'on réalise des animations un peu complexes ou même des jeux, la fonction de détection de collisions va nous permettre de détecter et donc de pouvoir réagir lorsqu'un objet en rencontre un autre au cours de son animation. Cette fonction s'appelle **hitTest()** et s'utilise ainsi :

```
this.hitTest(_root.occurrence) {  
    action  
}
```

Root représente votre scène. Cette ligne signifie que lorsque « l'occurrence » va entrer avec en collision avec l'objet implémentant cette méthode, action sera réalisée.

Examinons l'exemple suivant :

Faisons 2 clips :

- 1 contenant le dessin d'une balle
- 1 contenant une barre représentant un mur

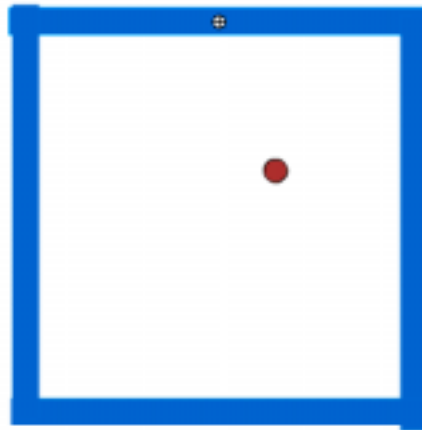


Figure 11 : Détection de collisions

Plaçons ensuite sur la scène 4 occurrences de l'objet mur afin de réaliser un carré..sorte de Trinquet ! Ajoutons ensuite une occurrence de la balle. On prendra soin de donner un nom à cette occurrence via les propriétés :

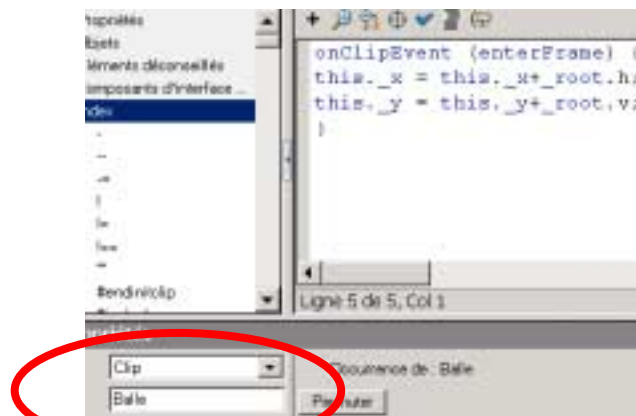


Figure 12 : Nommage d'occurrences

A la frame 1 de la scène, nous allons initialiser 2 variables H et V = 7 (H=7; V=7;)

Activons la balle, et ajoutons la suite d'instructions suivante :

```
onClipEvent (enterFrame) { //Cette partie de code va s'exécuter en boucle lors du chargement du clip.
```

```

    this._x = this._x+_root.h; // root représente la scène, root.h et root.v permet de récupérer le contenu des variables
    H et V précédemment déclarées. This représente l'accès à l'objet lui même, c'est à dire, ici , la balle et ses
    coordonnées.
    this._y = this._y+_root.v;
}

```

Avec ce code nous créons une balle qui va de décaler en diagonale vers le bas à droite.

Il nous reste à détecter la collision avec chaque instance du mur et de modifier le déplacement en conséquence :

Mur Gauche	Mur Droit	Mur Bas	Mur Haut
<pre> onClipEvent (enterFrame) { if (this.hitTest(_root.balle)) { _root.h = 5; } } </pre>	<pre> onClipEvent (enterFrame) { if (this.hitTest(_root.balle)) { _root.h = -5; } } </pre>	<pre> onClipEvent (enterFrame) { if (this.hitTest(_root.balle)) { _root.v = -5; } } </pre>	<pre> onClipEvent (enterFrame) { if (this.hitTest(_root.balle)) { _root.v = 5; } } </pre>

Et hop...le mouvement perpétuel est crée !

9 Détection de « clics souris » et duplication de dessins (exo10.fla)

Nous allons créer une animation qui va nous permettre de détecter des clics souris et de dupliquer à l'endroit cliqué une occurrence de symbole de type Clip.

Dans un premier temps, nous allons créer et placer sur la scène un symbole de type clip (un rond noir par exemple). Nous créons ensuite un deuxième symbole de type bouton qui va recouvrir tout ou partie de la scène mais qui sera transparent. L'occurrence de la balle sera nommée : Rond

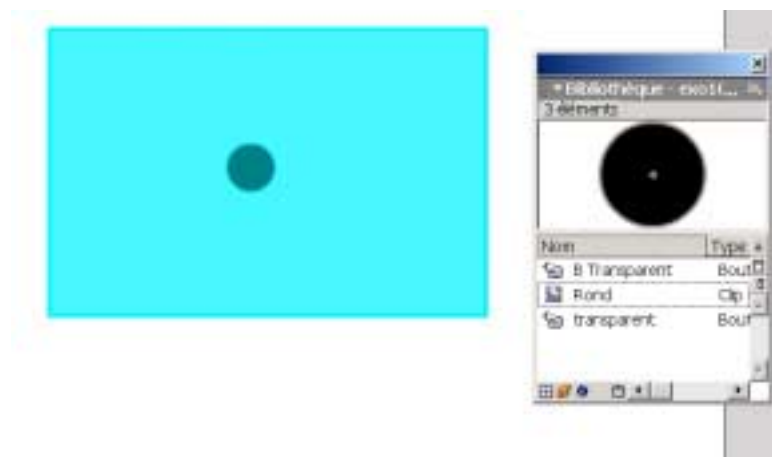


Figure 13 : Détection de clics et duplication de symboles

Nous allons ensuite ajouter le code suivant au bouton transparent :

```

on (press) {
    n=n+1;
    duplicateMovieClip ( "Rond", "Rond"+n, n );
    eval("Rond"+n)._x=_xmouse
    eval("Rond"+n)._y=_ymouse
}

```

La fonction `duplicateMovieClip` permet de dupliquer les animations. Le premier paramètre est le nom de l'occurrence à dupliquer, le second est le nom que l'on va donner à la nouvelle occurrence ; ici, elles seront toutes du style : Rond1, Rond2, ...

La fonction *Eval* permet d'accéder aux variables, propriétés, objets et clips par leur nom. Si l'expression est un objet ou un clip, une référence à l'objet ou au clip est renvoyée. Si l'élément nommé dans l'expression est introuvable, *undefined* est renvoyé. Les champs `_x` et `_y` de chaque occurrence nouvelle dupliquée et positionnée à la valeur du pointeur de la souris.

10 Déplacement interactifs de clips (exo11 fla)

Nous allons maintenant reprendre le clip du rond précédent et ajouter le code permettant de le déplacer librement par glisser/déplacer.

Nous allons dans un premier temps créer 1 ronds de type clip et associer y le code suivant :

```
onClipEvent (mouseDown) {  
  startDrag (this);  
}  
onClipEvent (mouseUp) {  
  stopDrag ();  
}
```

Et hop, on peut déplacer librement le rond.

Nous allons maintenant créer un symbole rond noir en tant que bouton et placer 2 occurrences sur la scène et les nommer *o1* et *o2*.



Figure 14 : Déplacements différents

Pour la première occurrence, on va y associer le code suivant :

```
on (press) {  
  startDrag (o1,false); // avec true, le symbole sera centré sur le pointeur de la souris.  
}  
on (release) {  
  stopDrag ();  
}
```

et pour la seconde

```
on (press) {  
  startDrag (this,false);  
}  
on (release) {  
  stopDrag ();  
}
```

Testez et observez les différences.

La fonction *startDrag* accepte d'autres paramètres :
startDrag(cible,[verrouiller ,gauche, haut, droite, bas])

Les valeurs relatives *gauche, haut, droite, bas* correspondent aux coordonnées du parent du clip spécifiant un rectangle de contrainte pour le clip. Ces paramètres sont facultatifs. Les contraintes obligeront le rond à ce déplacer uniquement dans le rectangle ainsi définit.

Des informations plus complètes sur chaque fonction sont disponibles dans *l'Aide/Dictionnaire Action Script*.

11 Déplacement dans un clip (exo12 fla)

Nous allons créer une animation qui va permettre de changer la couleur d'un clip de manière interactive. Nous allons pour cela créer un clip contenant un rectangle.

Nous allons ensuite créer 3 images clés supplémentaires avec ce rectangle, mais d'une couleur différente à chaque fois.

Sur la première image clé, nous ajouterons le code suivant :

Stop ()

Ceci évitera à l'animation de démarrer et de voir successivement les 4 rectangles différents.

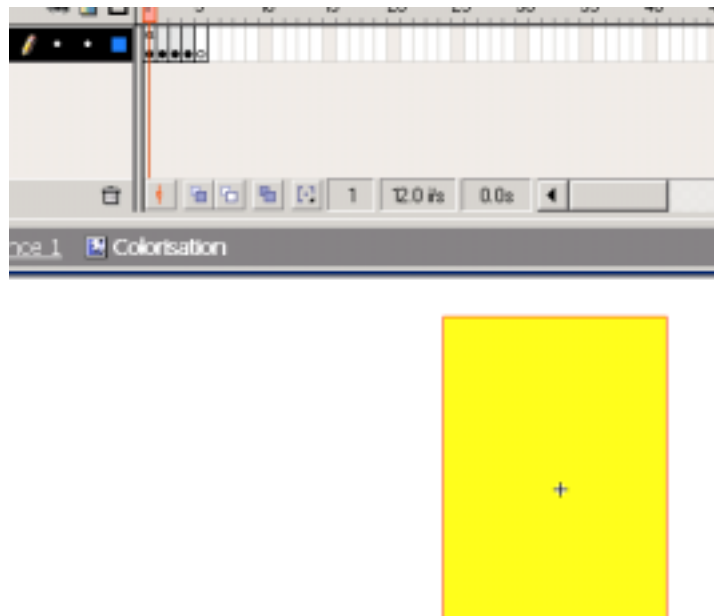


Figure 15 : Figure dupliquée

Nous allons ensuite créer 4 boutons correspondant aux couleurs des rectangles créés et placer tout ce petit monde sur la scène.

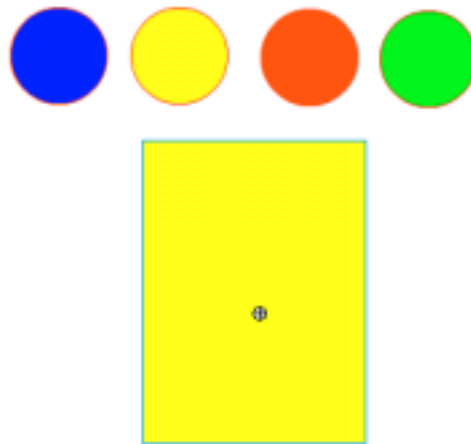


Figure 16 : Interface

Enfin, nous associons le code suivant à chaque bouton :

```
on (press) {
  dessin.gotoandstop (1)
}
```

Le numéro dans le *gotoandstop* correspond à l'image clé que l'on veut afficher. Elle changera pour chacun des boutons de manière à correspondre à la couleur désirée.

12 Gestion de l'interactivité

12.1 Clavier (exo13.fla)

On crée deux zones de textes, une de type texte de saisie (à spécifier dans les propriétés) et l'autre dynamique. On pourra leur donner un label avec des zones de texte statiques. La première zone aura comme nom de variable *prenom*, la seconde *voir* (cf. propriétés). On créera ensuite un bouton OK qui permettra lors de mettre à jour la zone dynamique avec le contenu de la zone de saisie. On initialisera les deux zones à « Entrez votre prénom » pour la première, vide pour la seconde.

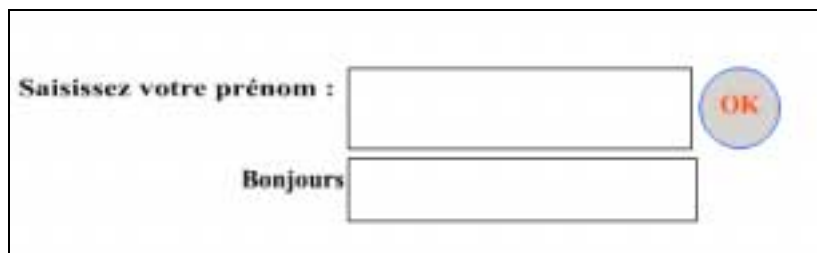


Figure 17 : Gestion des zones de texte

Afin de gérer le clic souris, on associera le code suivant au bouton :

```
on (press) {
  voir = prenom;
}
```

et on initialisera les variables avec les valeurs nécessaires.

12.2 Souris – Coordonnées (exo14 fla)

On va réaliser un afficheur de coordonnées de souris. Pour cela, on va créer 2 zones de texte dynamiques associées aux variables X et Y et avec le label correspondant.



On va transformer ces zones de texte en clip de manière à accéder aux méthodes liées aux clips (gestion de la souris notamment).

On y associera le code suivant :

```
onClipEvent (enterFrame) {  
    X = _root._xmouse;  
    Y = _root._ymouse;  
}
```

Ce composant « compteur souris » pourra être réutiliser sans aucun problème dans n'importe quelle animation.

12.3 Souris – Drag and Drop (exo15 fla)

On va créer un clip représentant un rond.. Ce clip sera placé sur la scène avec comme nom d'occurrence « rond » et comme type d'occurrence « bouton » nous permettant ainsi de gérer les événements press (*appuyé*) et release (*lâché*)
On associera à ce bouton le code suivant :

```
on(press) {  
    startDrag("rond");  
}  
  
on(release) {  
    trace(_root.rond._y); // trace affiche sur la console de sortie les coordonnées du rond  
    trace(_root.rond._x);  
    stopDrag();  
}
```

13 Composants d'interface (exo16 fla)

La version MX de Flash propose un ensemble de « widgets » ou composants d'interface permettant la réalisation de formulaires.

Nous allons présenter ici un exemple permettant la sélection d'un nom dans une liste et nous allons générer son adresse mail en fonction de son université d'appartenance.

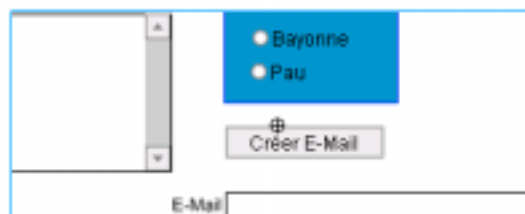


Figure 18 : formulaire de saisie

Nous allons regrouper ces composants en *movieclip* de manière à exécuter les initialisation lors de son chargement - *onClipEvent (load)* – Dans cette initialisation, nous allons créer le contenu de la liste (également faisable de manière interactive) :

```
onClipEvent (load) {
    extension="";

    noms.setsize(100,100);
    noms.additem("Philippe","Philippe.Roose");
    noms.additem("Michel","Michel.Filbet");
    noms.additem("Marc","Marc.Dalmau");

    function creermail() {
        if (bayonne.getstate() == true)
            extension = "@iutbayonne.univ-pau.fr" ;
        if (pau.getstate() == true)
            extension = "@univ-pau.fr" ;

        mail=noms.getValue()+extension;
    }
}
```

La fonction *creermail()* sera associée à l'attribut « clickhandler » de l'objet bouton. Les deux boutons radio seront regroupés sous le nom *universite* et auront comme nom d'occurrence *bayonne* et *pau*. La liste sera appelée *noms*.

14 Gestion des variables par fichiers

Il est également aisé d'interagir avec des fichiers. La fonction s'appelle *loadVariables(URL relative|absolue, location, variables)*. L'URL contient un chemin absolu ou relatif vers un fichier. La location sera l'endroit où l'on veut stocker les variables (*_root* les place au niveau global, *this* les place dans le composant concerné). La syntaxe des fichiers doit être relative au passage de paramètres selon le protocole HTTP. Aussi, si l'on utilise la méthode POST et que l'on veut récupérer les variables *nom* et *mail* contenu dans un fichier, on aura :

```
loadVariables("fichier.txt",this,"POST");
```

Le contenu du fichier aura la syntaxe suivante :

&nom=Philippe&mail=Philippe.Roose@iutbayonne.univ-pau.fr

Attention, comme en HTML, le contenu des variables sera forcément considéré comme du texte. Si l'on désire l'utiliser en tant que numérique par exemple, il faudra changer son type avec par exemple la fonction *number* :

```
Age=number(Age) ;
```

On modifiera l'exemple précédent en positionnant une valeur de mail par défaut contenue dans un fichier.

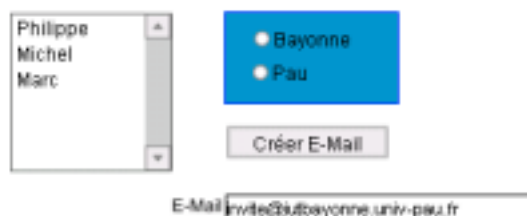


Figure 19 : Récupération de variables par fichiers

15 Conclusion

Nous y sommes (enfin diront certains !). C'est la fin du cours, mais quelques précisions afin d'éviter de mal comprendre tout ceci.

Tout d'abord, ce cours n'a pas la prétention d'être fait par et pour des experts. Il n'a pas non plus la prétention de se vouloir exhaustif, la quantité de possibilités offertes par Flash MX et telle que le double ou le triple des séances n'y suffiraient pas. Nous n'avons par exemple pas vu les interactions entre Flash, PHP et XML. La vision qui y est faite ici est celle d'un informaticien, et non celle d'un graphiste. Aussi, vous excuserez la pauvreté « visuelle » des exemples. Néanmoins, ce cours a deux objectifs :

- Vous introduire à la programmation de Flash par l'interface elle-même mais également par le langage associé *actionScript*.
- Ouvrir une fenêtre sur l'étendue des possibilités du langage et vous donnant toutes les bases afin de VOUS permettre de voir l'étendue des possibilités et ainsi d'avoir les clés pour des développements conséquents par le futur.